Web Systems Development

What a Framework Gets you: Laravel

RPI

Agenda

THIS WEEK'S READING

The Eloquent docs, and one optional book

REQUIRED

Laravel docs: Eloquent ORM

- Before today: Basic Usage, Mass Assignment, and Insert, Update, Delete
- Before Friday: Relationships, Querying Relations, Eager Loading
- Skip for now: soft deletes, scopes, polymorphic relations, events

OPTIONAL

Laravel: Up & Running, ch. 5

- Matt Stauffer, 3rd ed. Through the RPI library: sign in with your RPI credentials
- Today: migrations, seeding and factories, the start of Introduction to Eloquent
- Friday: Eloquent relationships, eager loading, and the testing section

Written for older Laravel versions than the 13 we run. Casts are a casts() method now, which is how the drills write them.

TOOLING · DOCKER COMPOSE + ARTISAN

Run Laravel's command line inside the app container

The prefix chooses a running Docker service; everything after the service name is the command executed inside that container.

```
docker compose exec app php artisan <command> [arguments] [--options]
```

Compose CLI	run in container	service	Laravel CLI	operation	optional inputs and flags

BEFORE IT WORKS

1 Start the services

```
docker compose up -d
```

2 Run from the project directory

Compose must find compose.yaml and the app service.

If the app container is not running, exec fails—it does not start a new service container.

REPLACE <command> WITH…

`list`	discover available Artisan commands
`help migrate`	show arguments and options for migrate
`migrate`	run pending database migrations
`tinker`	open Laravel's interactive PHP shell

CAUTION `migrate:fresh --seed` **drops every table, rebuilds the schema, then seeds data—development/test only.**

Sources: Docker Compose exec reference · Laravel 13.x Artisan console documentation

WHAT IS A DATABASE? IS IT THE SAME AS A DATABASE MANAGEMENT SYSTEM (DBMS)?

What is a database?

A database is a digital repository for storing, managing and securing organized collections of data.

https://www.ibm.com/think/topics/database

Authors



Matthew Kosinski

Staff Editor

IBM Think

Database vs. Database Management System (DBMS)

A Database and a Database Management System (DBMS) are closely related terms, but they serve different purposes:

- **A database is a structured set of data**. The data can be structured or unstructured and stored in various formats like tables, documents, and key-value pairs. It could be anything from a simple shopping list to a picture gallery or the vast amount of information in a corporate network.
- **A DBMS (Database Management System) is software used to interact with a database**. It provides an interface for users or applications to manipulate data, making the handling of large amounts of data more efficient and less error-prone. A DBMS oversees core administrative tasks such as data storage, retrieval, security, concurrency control, backup and recovery, and query processing. Examples include Oracle Database, MySQL, Microsoft SQL Server, and MongoDB.

Source: https://rivery.io/data-learning-center/database-types-guide/

WHAT TYPES OF DATABASES ARE USED FOR WEB APPLICATIONS?

DATABASES FOR WEB APPLICATIONS

Choose by workload: integrity, flexibility, speed, or relationship depth

The right question is not “SQL or NoSQL?” It is “What must this workload do reliably, at the scale and latency users expect?”

01 · SHAPE

How is the data organized?

Tables, JSON documents, key–value pairs, relationships, time-stamped events, geospatial objects, or files.

02 · ACCESS

How will the application retrieve it?

Joins, primary-key lookups, text relevance, graph traversal, time windows, nearby-location queries, or object download.

03 · GUARANTEE

What cannot fail?

Strong transactions, horizontal availability, sub-millisecond response, durable retention, global distribution, or flexible evolution.

Practical rule: Start with one durable system of record; add specialized stores only when a measured access pattern justifies them.

Adapted from Dinesh Arney, “Choosing the Right Database,” May 31, 2025.

THE LANDSCAPE

Most web applications need a portfolio, not one universal database

SYSTEM OF RECORD Durable business entities and application state	**Relational** Tables, joins, ACID transactions. POSTGRESQL · MYSQL	**Document** Flexible JSON records and evolving schemas. MONGODB · COUCHBASE	**Spatial** Geometry, proximity, containment, and routes. POSTGIS · SQL SERVER
SPEED & SCALE High-volume access and low-latency state	**Key–value** Direct lookup by key at massive throughput. DYNAMODB · REDIS	**In-memory** RAM-first caching, sessions, counters. REDIS · MEMCACHED	**Wide-column** Distributed writes and query-first partitions. CASSANDRA · BIGTABLE
SPECIALIZED QUERY Purpose-built retrieval and analysis	**Full-text search** Relevance, typo tolerance, language analysis. ELASTICSEARCH · SOLR	**Graph** Deep relationships, paths, network patterns. NEO4J · NEPTUNE	**Time-series** Fast ingestion and time-window aggregation. TIMESCALEDB · INFLUXDB
UNSTRUCTURED DATA Large binary objects and inexpensive retention	**Blob / object storage** Images, video, documents, backups, data lakes. S3 · AZURE BLOB · GCS	**Vector database** Embeddings for semantic search, similarity retrieval, and RAG models. PINECONE WEAVIATE MILVUS	**Deliver through a CDN** Optimize hot content for global download and streaming.

Categories overlap: Redis is key–value and in-memory; PostgreSQL can support spatial and time-series workloads.

RELATIONAL · SYSTEM OF RECORD

Relational databases remain the safest default for business transactions

ONE CHECKOUT · MULTIPLE CONSISTENT CHANGES

Order
Create the order and line items with a stable identifier.

Inventory
Reserve stock without overselling the same units.

Payment
Record authorization and commit—or roll back—together.

ATOMIC
All steps succeed or none do.

CONSISTENT
Rules remain valid after writes.

ISOLATED
Concurrent work does not collide.

DURABLE
Committed data survives failure.

Use it when...
- data is structured and relationships matter
- queries need joins, filters, and reporting
- multi-row or multi-table transactions are critical
- the database is the authoritative source of truth

Design for the trade-offs
- define and evolve the schema deliberately
- index around real query patterns
- use replicas, partitioning, or distributed SQL when scale demands it

Common choices: PostgreSQL, MySQL, Microsoft SQL Server, Oracle Database.

FLEXIBILITY & LATENCY

Document, key–value, and in-memory stores make different speed–flexibility trades

RICHER QUERY & STRUCTURE

FASTER DIRECT ACCESS

Document

Stores JSON-like records and understands nested fields, arrays, and field-level indexes.

BEST FOR

Product catalogs, content, profiles, event records, and rapidly evolving schemas.

MongoDB · Couchbase · CouchDB · Cosmos DB

Key–value

Retrieves an opaque value by a known key; the application owns most query logic.

BEST FOR

Sessions, user preferences, feature flags, counters, and high-throughput state.

DynamoDB · Riak KV · Redis

In-memory

Keeps the working dataset in RAM to minimize latency; may also persist or replicate.

BEST FOR

Caching, rate limits, leaderboards, transient workflow state, and traffic spikes.

Redis · Memcached · Apache Ignite · SAP HANA

Do not confuse the categories: key–value describes the data model; in-memory describes where the working data lives. Redis is both.

Core design questions: cache invalidation, shard key quality, value size, consistency level, memory cost, and persistence requirements.

Search and vector indexes belong beside the primary database—not in place of it

Primary database
Owns products, articles, permissions, and transaction truth.

Change pipeline
Events, change data capture, or scheduled indexing keep the copy current.

Search + vector indexes
Inverted indexes rank keywords; embeddings retrieve semantic neighbors for hybrid search and RAG.

Example: a user asks a natural-language question; vector search retrieves relevant documents, which a RAG model uses to ground its answer.

Add search when users expect…

- keyword and phrase search
- semantic similarity search
- relevance ranking and filters
- RAG answers grounded in content
- fast log exploration

Engineer explicitly for…

indexing cost, chunking, embedding model choice, hybrid ranking, authorization, freshness, and evaluation of retrieved context.

Common choices: Elasticsearch, OpenSearch, Pinecone, Weaviate, Milvus, pgvector.

SPECIALIZED RELATIONSHIPS

Graph and spatial databases win when the question is about connections or proximity

Graph database

“How are these entities connected?”

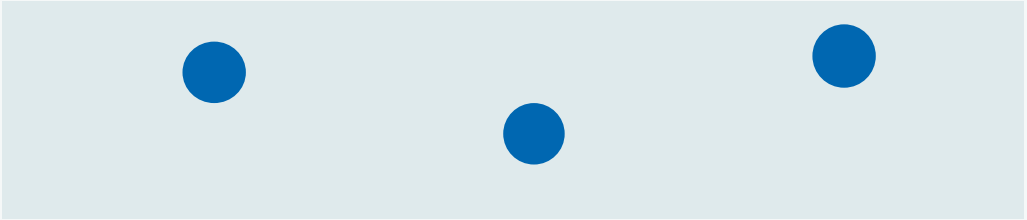


Nodes and edges are first-class data. Use graph traversal for recommendations, fraud rings, dependency analysis, knowledge graphs, and network paths.

Neo4j · Amazon Neptune · JanusGraph · TigerGraph

Spatial database

“What is nearby, inside, intersecting, or on a route?”



Geometry types and spatial indexes support nearby search, GIS, delivery zones, asset tracking, spatial joins, and map-based experiences.

PostGIS · Oracle Spatial · SQL Server Spatial · SpatiaLite

Design around traversal depth, graph partitioning, coordinate systems, geometric precision, and index strategy.

SCALE-SPECIALIZED STORES

Wide-column, time-series, and blob storage address three distinct scale patterns

Wide-column Cassandra · HBase · ScyllaDB · Bigtable	Distributes sparse, high-volume rows across many nodes. Model tables around known queries, partition keys, and write paths—not ad hoc exploration.	Use for: clickstreams, telemetry, global event logging, high write throughput. Watch: hot partitions, compaction, row width, eventual consistency.
Time-series TimescaleDB · InfluxDB · Prometheus	Optimizes append-heavy, timestamped measurements and range aggregates. Retention, compression, downsampling, and tag cardinality are core design decisions.	Use for: infrastructure metrics, IoT sensors, market data, usage analytics. Watch: late data, ingestion batching, retention cost, high-cardinality tags.
Blob storage Amazon S3 · Azure Blob · GCS · MinIO	Stores large opaque objects cheaply and durably. Keep searchable metadata and permissions in a database; use object keys and URLs to retrieve bytes.	Use for: images, video, documents, backups, archives, data lakes. Watch: egress cost, access control, lifecycle tiers, multipart transfer.

These systems are complementary: telemetry may land in a wide-column store, aggregate in a TSDB, and archive raw files in object storage.

POLYGLOT PERSISTENCE

A modern web application may use different databases

Browser / client
Requests pages, data, uploads, and search.

→

Web application
Owns business rules, authorization, and orchestration.

→

Relational primary
Authoritative users, orders, permissions, and payments.

Cache & sessions
Reduce repeated work and absorb traffic spikes.

REDIS

Search index
Serve ranked text queries and faceted navigation.

OPENSEARCH

Object storage
Keep uploads and media outside the transactional DB.

S3 / BLOB

Events & metrics
Analyze time-series and high-volume events.

TIMESCALE / CASSANDRA

Keep the architecture governable

One source of truth
Define which store owns each entity.

Explicit synchronization
Use events or change data capture for derived stores.

Failure-aware reads
Plan for stale or unavailable cache, search, and analytics.

Add a datastore because it solves a demonstrated workload—not because it is fashionable.

DECISION CHECKLIST

Make the database choice with six design questions

01 · What is the dominant access pattern?
Joins, key lookup, document query, text ranking, traversal, time range, spatial relation, or object retrieval?

02 · What consistency is required?
Strong ACID transactions, tunable consistency, or acceptable eventual consistency?

03 · How will the data grow?
Rows, documents, writes per second, object size, graph density, or time-series cardinality?

04 · How stable is the schema?
Stable schema, flexible attributes, new event types, or frequently changing content?

05 · What latency and availability matter?
Interactive milliseconds, cache microseconds, global uptime, offline sync, or batch analytics?

06 · Can the team operate it well?
Skills, monitoring, backups, recovery, cost controls, migrations, and managed-service maturity.

Reasonable defaults

Start: PostgreSQL or MySQL for core application state.

Add Redis: after measuring repeated reads, sessions, counters, or latency pressure.

Add search: when relevance and text experience exceed SQL search.

Add a specialist: only when graph, spatial, time-series, massive-write, or file workloads are central.

Final principle: the simplest architecture that meets integrity, scale, latency, and operability requirements is usually the best architecture.

Source: Dinesh Arney (2025), adapted and condensed for web application design.

DATABASES AND LARAVEL

LARAVEL DATABASE BASICS

- MariaDB 10.3+ (Version Policy)
- MySQL 5.7+ (Version Policy)
- PostgreSQL 10.0+ (Version Policy)
- SQLite 3.26.0+
- SQL Server 2017+ (Version Policy)

All of these are supported. Notice all relational.

LARAVEL · DATABASE CONNECTIONS

One file defines the connections; .env selects the active one

Laravel ships connection definitions for supported drivers in config/database.php. Environment variables provide the machine-specific credentials.

1 · ENVIRONMENT VALUES

Project root: .env

```
DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=pantry
DB_USERNAME=app
DB_PASSWORD=••••••
```

.env.example documents the required keys; .env holds local secrets and should not be committed.

reads

2 · CONFIGURATION MAP

config/database.php

```
'default' => env(
  'DB_CONNECTION', 'sqlite'
),

'connections' => [
  'sqlite' => [...],
  'mysql'  => [...],
  'pgsql'  => [...],
  'sqlsrv' => [...],
],
```

Named connections translate environment values into driver options.

feeds

3 · APPLICATION USE

Framework data APIs

Eloquent models
Query Builder
DB facade
Migrations
Transactions

```
Default:
DB::table('products')

Named:
DB::connection('pgsql')
```

KEY DISTINCTION

Laravel defines several named driver configurations—but DB_CONNECTION chooses the default connection used by Eloquent and the DB facade.

Official documentation https://laravel.com/docs/13.x/database#configuration

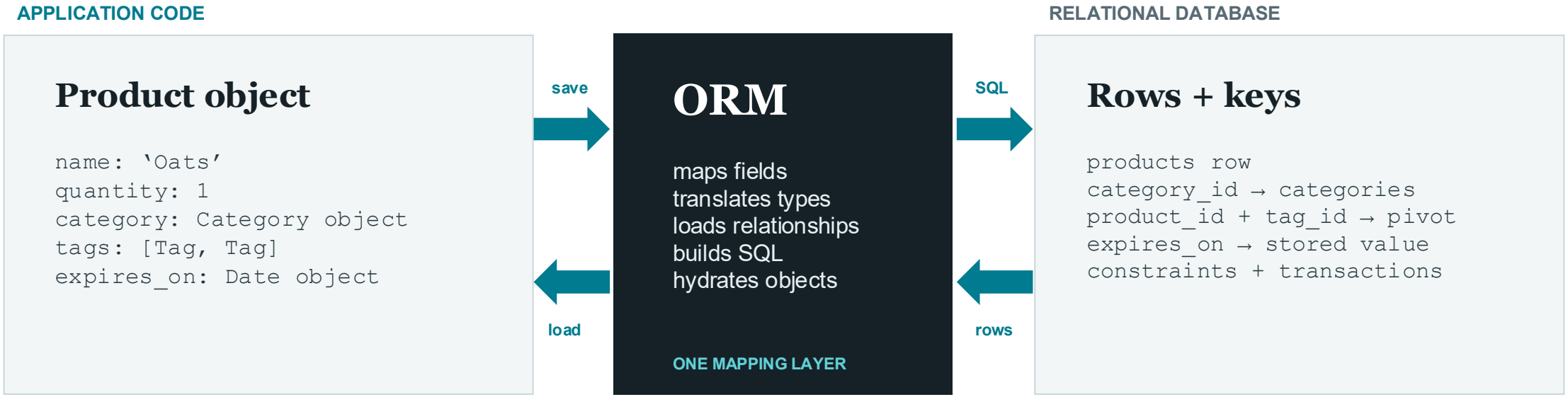
After changing production environment values: php artisan config:cache

WHAT PROBLEM DOES ORM SOLVE?

THE ANSWER

ORM centralizes the translation between objects and relational tables

It does not eliminate the impedance mismatch; it automates the repetitive mapping and persistence code the mismatch creates.



THE PROBLEM IT SOLVES

Hand-written CRUD SQL **Row ↔ object conversion** **Relationship joins** **Type conversion**

The ORM makes persistence a reusable convention instead of repeated translation code scattered throughout the application.

ORM manages the mismatch; good design still requires understanding the SQL, relationships, indexes, and transactions underneath it.

ORM + WEB FRAMEWORKS

A framework can make one ORM the default—or treat persistence as a replaceable adapter

Coupling describes how strongly framework conventions, tooling, and application structure assume a particular persistence layer.

TIGHT COUPLING

The ORM is on the paved road



Framework features assume the ORM

Models, relationships, migrations, validation, CLI generators, and request-to-model conventions are designed to work together.

Fast defaults; less wiring. Replaceing the ORM usually means leaving the framework's conventions and rewriting application code.

EXAMPLES Laravel / Eloquent · Rails / Active Record · Django / Django ORM

LOOSE COUPLING

Persistence plugs into the framework



The framework owns the lifecycle—not the mapping

Routing, dependency injection, and transactions can call an interface or repository while the application selects the ORM or SQL tool.

More choice and easier substitution; more configuration, integration decisions, and responsibility for consistent patterns.

EXAMPLES Spring / Hibernate or jOOQ · Express / Prisma, TypeORM, Sequelize

DO NOT CONFUSE THE TWO AXES

Tight vs. loose describes framework integration. Active Record vs. Data Mapper describes where persistence logic lives.

Illustrative mapping: Laravel = tight + Active Record; Spring/Hibernate = loose + Data Mapper. Sources: official Laravel, Rails, Spring, and Express documentation.

THE WEEK'S QUESTION

One product, two shapes

IN YOUR CODE — ONE OBJECT

```
$oats->name             // 'Oats'
$oats->quantity         // 1
$oats->category->name   // 'Grains'
$oats->tags             // [staple, organic]
$oats->expires_on       // a date object

// follow any arrow, any time
```

IN THE DATABASE — FOUR ROWS, THREE TABLES

```
products
 id | name | quantity | category_id | expires_on
  2 | Oats |        1 |           1 | 2027-01-17

categories       product_tag
 id | name        product_id | tag_id
  1 | Grains               2 |      1
                           2 |      2
```

A reference becomes a number. A list becomes a table. A date becomes a string. Something has to translate — every time, in both directions — and that something has opinions.

The thing that translates is an object-relational mapper: an ORM. Laravel's is called Eloquent.

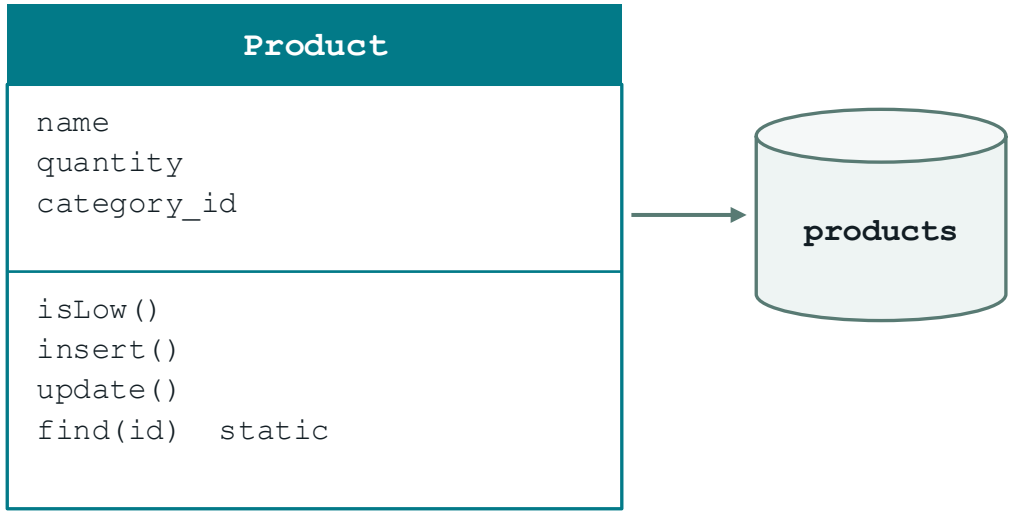
What objects assume that tables don't

	An object has…	A table has…
References	a property that points at another object	a number in a foreign-key column
Collections	an array inside the object	a separate table, one row per pairing
Types	dates, booleans, other objects	whatever the engine stores — often a string
Identity	$a === $b: the same object in memory	a primary key: two rows with id 2 are one row
Loading	walk to any related object, for free	every hop is another query
Inheritance	class Tinned extends Product	no such thing

“Impedance mismatch” is borrowed from electrical engineering: two parts that don't fit, so energy is lost where they join.

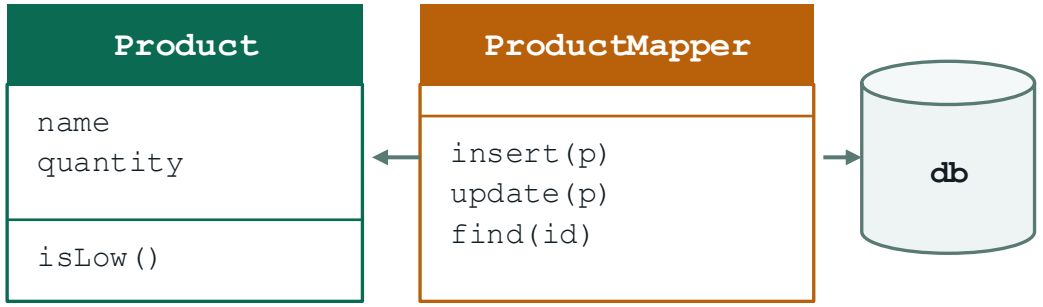
Two ways to build the translator

ACTIVE RECORD



The object wraps one row. It carries the data, the rules and the SQL, and saves itself.

DATA MAPPER



The object knows nothing about storage. A separate layer moves data between it and the database.

Look at where insert(), update() and find() live. That's the whole difference.

THE TRADE

What each pattern costs

	Active Record	Data Mapper
Used by	Eloquent, Rails, Django	Doctrine, Hibernate, week 3's Repository
To start	an almost empty class	a mapper (or mapping config) for every type
Your objects	are rows: they extend Model, one property per column	are plain PHP; they know nothing about storage
A schema change	shows up in the object at once	is absorbed by the mapper; objects can stay put
Testing the rules	needs a database (Friday: RefreshDatabase)	runs with no database at all — week 3's unit tests
Fits best when	objects look like the tables	objects and tables have drifted apart

Fowler's advice, roughly: Active Record is a good choice while the logic is simple and objects mirror tables. The question is what you do when they stop.

ACTIVE RECORD · PATTERN

The object is the row—and saves itself

THE MODEL COMBINES THREE RESPONSIBILITIES

```
STATE
name, quantity, category_id

DOMAIN BEHAVIOR
isLow(), restock()

PERSISTENCE
find(), save(), delete()
```

FOLLOW ONE OBJECT THROUGH ITS LIFECYCLE

```
1  Product::find(2)
   SELECTS a row and creates Product

2  $product->restock(3)
   changes state and applies a rule

3  $product->save()
   UPDATES the row represented by this object
```

WHY IT WORKS

The same object is both the business object and persistence gateway. CRUD stays short, conventions remove mapping configuration, and teams move quickly.

TRADE-OFF

The class is coupled to the table. Schema changes affect object properties, and persistence-heavy model tests usually require a database.

Best fit: straightforward CRUD applications where objects closely mirror tables. Laravel Eloquent and Rails Active Record use this pattern.

DATA MAPPER · PATTERN

Domain objects stay clean; a mapper handles storage

THE DOMAIN OBJECT KNOWS BUSINESS RULES

```
final class Product {
  private int $quantity;

  public function restock(int $units): void {
    $this->quantity += $units;
  }
}

// no SQL, table name, or save()
```

THE MAPPER / REPOSITORY KNOWS STORAGE

```
$product = $repo->find(2);
// SELECT + row → Product

$product->restock(3);
// pure domain behavior

$repo->save($product);
// Product → columns + UPDATE
```

WHY TEAMS ACCEPT THE EXTRA LAYER

The domain can differ from the schema, combine multiple tables, and be unit-tested without a database. The mapper absorbs storage changes.

TRADE-OFF

Each entity needs mapping code or configuration. Object identity, relationships, and transaction boundaries require more architecture.

Best fit: complex business domains where object design should evolve independently of storage. Doctrine and Hibernate use Data Mapper; Repository is a common façade.

ELOQUENT IS ACTIVE RECORD: The class is the row

app/Models/Product.php — THE STARTER, ALL OF IT

```
class Product extends Model
{
    use HasFactory;

    protected $fillable = ['name', 'quantity',
        'category_id', 'expires_on'];
}

// no table name, no columns, no SQL
```

WHAT IT CAN ALREADY DO

```
$oats = Product::find(2);    // SELECT … id = 2
$oats->quantity = 3;
$oats->save();               // UPDATE … SET

Product::create(['name' => 'Rice',
                 'quantity' => 2]);  // INSERT
$oats->delete();             // DELETE
```

- Class Product reads table products — the plural, in snake_case. Every column is a property, undeclared.
- id is the primary key; created_at and updated_at are kept for you. $fillable lists the columns create() may set.

LARAVEL · database/ DIRECTORY

Three folders make the database reproducible

Migrations define structure. Factories define believable model data. Seeders decide what data should actually be inserted.

1 `database/migrations`

Build the schema

What shape should the database have?

- Create and alter tables
- Define columns, keys, and constraints
- Ordered by timestamp
- Versioned with the application

```
php artisan migrate
```

2 `database/factories`

Define sample models

What does a believable Product look like?

- Default fake attributes
- Named states and relationships
- Reused by tests and seeders
- Writes rows only when create() runs

```
Product::factory(20)->create()
```

3 `database/seeders`

Assemble a dataset

Which records should exist after setup?

- DatabaseSeeder is the entry point
- Inserts fixed reference data
- Calls factories for bulk sample data
- Controls creation order

```
php artisan db:seed
```

THE COMMON RESET FLOW

`migrate:fresh --seed` drop tables → migrate → seed → factories create rows

Development/test only: migrate:fresh destroys every existing table before rebuilding the database.

Mental model: migrations = structure · factories = recipes · seeders = scenarios

MIGRATIONS

The schema, as versioned code

WEEK 3 / HOMEWORK 1 — SQL RUN ONCE

```
// Repository::__construct()
$this->db->exec('CREATE TABLE IF NOT EXISTS
  products (
    id   INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE, ...
)');

// Homework 1: init.sql -- runs only on
// an empty volume. Change it, and nobody
// who already has data sees the change.
```

LARAVEL — 2026_10_20_000002_create_products_table.php

```
public function up(): void {
  Schema::create('products', function ($table) {
    $table->id();
    $table->string('name');
    $table->unsignedInteger('quantity')
          ->default(0);
    $table->foreignId('category_id')
          ->nullable()->constrained();
    $table->timestamps();
  });
}
public function down(): void {
  Schema::dropIfExists('products');
}
```

- One file per change, named with a timestamp, run in file-name order. Laravel records each one in a table called migrations, so each runs once per database.
- It lives in git with the code: a teammate pulls, runs migrate, and their tables match yours.

php artisan, in the app container

Command	What it does
`make:migration add_expires_on`	writes a new, empty migration file with today's timestamp
`migrate`	runs every migration not yet listed in the migrations table, as one batch
`migrate:status`	lists every migration file and whether it has run
`migrate:rollback`	runs down() on the last batch, newest first
`migrate:fresh --seed`	drops every table, runs every migration, then the seeder. Development only.

```
docker compose exec app php artisan migrate:fresh --seed
```

You'll run that line a lot today: after any change to a migration, the browser's MySQL tables are stale until you do.

MIGRATIONS · IN PRODUCTION

Why “down” is usually a lie

```
// 2026_11_02_000001_add_expires_on_to_products.php
public function up(): void {
  Schema::table('products', function ($table) {
    $table->date('expires_on')->nullable();
  });
}

public function down(): void {
  Schema::table('products', function ($table) {
    $table->dropColumn('expires_on');
  });
}
```

- down() reverses the schema, not the data. Roll this back on a live site and every expiry date is gone. up() brings back an empty column.
- So teams treat production migrations as forward-only: a mistake is fixed by a new migration, not by rolling back.
- And never edit a migration that has already run somewhere else — it won't run there again. Drill 1 breaks that rule on purpose: migrate:fresh rebuilds a database nobody cares about.

The schema is easy to reverse. The data in it isn't. A migration is a change to a database that already has someone's data in it.

SEEDERS AND FACTORIES

Test data, as code

A SEEDER — THE SAME PANTRY FOR EVERYONE

```
// database/seeders/DatabaseSeeder.php
$grains = Category::create(['name' => 'Grains']);
// name, quantity, category, days to expiry
['Rice', 2, $grains,  200, [$staple]],
['Oats', 1, $grains,   90, [$staple, $organic]],
['Beans', 5, $tinned, null, [$staple]],
// ...nine products in all
```

A FACTORY — A RECIPE; SAY ONLY WHAT MATTERS

```
// tests/Feature/Drills/Drill4Test.php
Product::factory()->create(
    ['name' => 'Walnuts', 'quantity' => 1]);
Product::factory()->create(
    ['name' => 'Quinoa', 'quantity' => 7]);

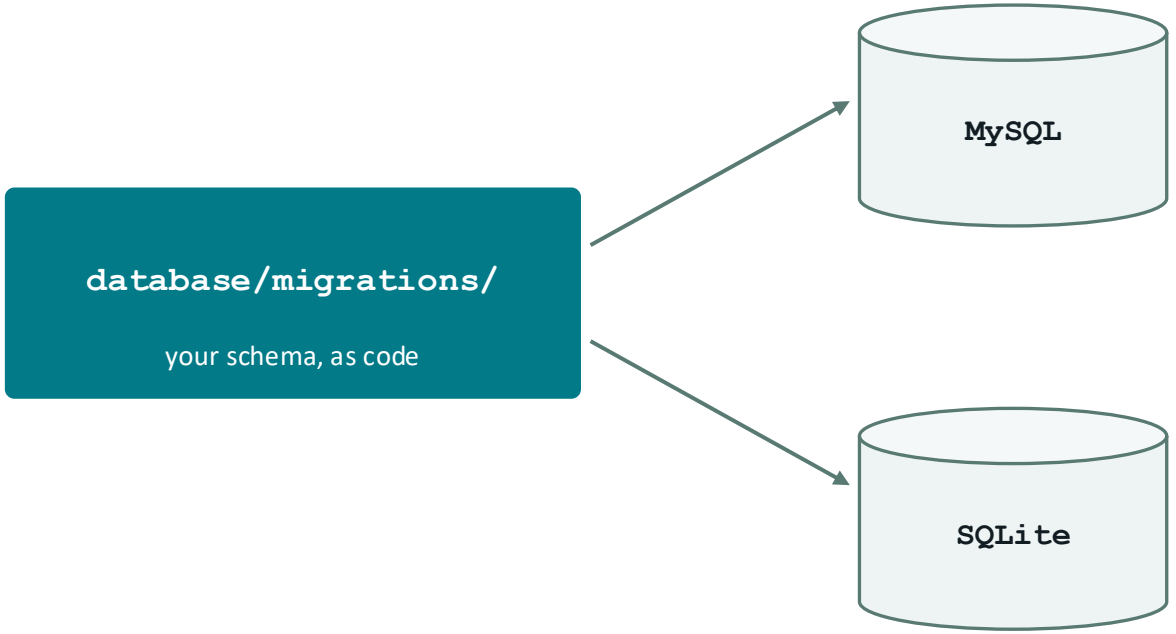
$this->get('/products/low')
    ->assertSeeText('Walnuts')
    ->assertDontSeeText('Quinoa');
```

- The seeder fills the database the pages show. migrate:fresh --seed puts everyone's pantry back to the same nine products.
- The factory makes rows for tests. It fills in a name and a category; the test states only the quantity, because that's what drill 4 is about.

Every test builds the rows it needs — the same rule as week 3's Playwright test naming its product Apples ${Date.now()}.

THE ONE CONFUSING THING TODAY

Two databases, one set of migrations



The pages you open
MySQL in the db container. Changes only when you run migrate:fresh --seed.

The Check result button
A fresh, empty database in memory, rebuilt from your migrations on every check.

So after drill 1 the check is green and the page hasn't changed. Same migrations, same models, two database engines, and nobody wrote SQL for either.

The honest limit: SQLite isn't MySQL. Tests that pass on one don't prove the app on the other. These checks avoid the differences.

TODAY'S DRILLS · 1–5

What you'd have written by hand

#	Week 3 / Homework 1	Eloquent
1	`edit init.sql; works on an empty volume only`	`$table->date('expires_on')->nullable();`
2	`SELECT * FROM products ORDER BY name, and a loop`	`Product::orderBy('name')->get()`
3	`if ($row === false) { http_response_code(404); }`	`Product::findOrFail($id)`
4	`a WHERE clause, values bound by hand with ?`	`Product::where('quantity', '<', 2)`
5	`new DateTime($row['expires_on']) everywhere`	`casts(): ['expires_on' => 'date']`

Drill 5 is the type row of the mismatch: the column holds a string, the cast hands you a date. ?->format() is for Beans, which never expires.

THE ORM WRITES SQL

Read what it wrote

```
Product::orderBy('name')->get()
  select * from `products` order by `name` asc

Product::findOrFail(2)
  select * from `products`
    where `products`.`id` = ? limit 1

Product::where('quantity', '<', 2)
        ->orderBy('name')->get()
  select * from `products`
    where `quantity` < ? order by `name` asc
```

- The ? is still there. Eloquent binds values the way week 3's prepare() did, so it's injection-safe by default.
- Product::all() and an @if in the view gives the same page. With nine products nobody can tell. With nine million, PHP fetches nine million rows to keep three.
- Drill 4's check rejects that version, and its message says why.

Every pantry page ends with “Queries this page ran”; every drill has “In plain SQL”, what you'd have written by hand. Compare them.

IN CLASS

Drills 1–5, in pairs

Migrations and models · 1–5

about 25 minutes

- 1 · a migration is the schema, as code
- 2 · a model reads the table
- 3 · find one, or 404
- 4 · let the database do the filtering
- 5 · casts: a column becomes a date

START IT

```
cd inclass/wk9/laravel
docker compose watch
# Watch enabled -> http://localhost:8000

# second terminal, after any migration change:
docker compose exec app \
  php artisan migrate:fresh --seed
```

Week 7 workshop still running? Stop it first: both use port 8000.

Hand-in: copy the laravel folder into inclass/wk9/ in your own repo and push it at the end of class — whatever you finished.

BEFORE FRIDAY

Read the relationships half of the docs

Tue 10/20
today: the mismatch, Active Record, migrations · drills 1–5

→

Eloquent docs
Relationships, Querying Relations, Eager Loading

→

Fri 10/23
relationships, the N+1 problem, database tests · drills 6–9

→

Fri 10/30
Homework 7 due: the to-do list on a database

- In the Eloquent docs: Relationships (one to many, many to many), Querying Relations and Eager Loading. Those are drills 6–9.
- Read Eager Loading for the problem it solves: one query for the list, then one more for every row. Friday you'll watch it happen.
- Optional: Stauffer ch. 5, the relationships section and the testing section at the end.
- Homework 6 is due Friday. Homework 7 is assigned Friday: the to-do list on a database.

Exit question: name one thing Eloquent did for you today that you'd otherwise have written in SQL — and one thing it did that you couldn't see.

REFERENCES

Where today's slides come from

Laravel documentation: Eloquent ORM. laravel.com/framework/docs/5.0/eloquent	*required*	**slides 7, 12–15**
Stauffer, M. (2023). Laravel: Up & Running (3rd ed.), ch. 5, Databases and Eloquent. O'Reilly. Through the RPI library. learning-oreilly-com.libproxy.rpi.edu/library/view/laravel-up/97810981…	*optional*	**slides 9–12**
Fowler, M. (2002). Active Record and Data Mapper. In Patterns of Enterprise Application Architecture. Addison-Wesley. martinfowler.com/eaaCatalog/	*cited, not assigned*	**slides 5–8**
Laravel 13.x documentation: Migrations, Query builder, Attribute casting, Seeding, Factories. laravel.com/docs/13.x/migrations	*for the drills*	**slides 9–15**

The first two are this week's reading. Drill code and teaching notes: inclass/wk9/laravel.